

11장 운영 구조 및 모니터링

(운영 인프라 및 관측성(Observability) 설계 부분)

11.8 장애 감지 및 자동 복구

메커니즘	팩트	이번 프로젝트 시스템은 1. 일부 컨테이너가 죽거나 2. 비정상 상태가 되어도 사람의 개입 없이 스스로 복구되도록 4가지 안전장치를 갖추고 있습니다.
restart 정책	restart: unless-stopped 12개 서비스 → 죽으면 자동 재시작	
Celery 큐 분리	alarm(concurrency=4) / metric(concurrency=1) → 알람 지연 격리	
헬스체크 의존	service_healthy 16곳 → 비정상 시 의존 서비스 대기	
Redis 영속	appendonly → 재시작 후 큐 데이터 유지	

① restart 정책 — 죽으면 자동으로 다시 살아난다

12개 서비스에 restart: unless-stopped 정책을 적용했습니다. 컨테이너가 비정상 종료(크래시)되면 Docker가 자동으로 재시작합니다. 다만 운영자가 의도적으로 멈춘 경우(docker stop)는 재시작하지 않아, 의도된 정지는 존중합니다. 실제로 Redis 프로세스를 강제 종료시켜도 약 8초 만에 자동 재기동되는 것을 확인했습니다.

② 헬스체크 의존 — 준비된 순서대로 켜진다

서비스 간 depends_on 에 condition: service_healthy 를 16곳 걸어두었습니다. 예를 들어 DRF는 PostgreSQL과 Redis가 “정상(healthy)” 판정을 받을 때까지 기다린 뒤 기동합니다. 이렇게 하지 않으면 DB가 아직 준비되지 않은 상태에서 앱이 먼저 떠 곧바로 죽는 문제가 생깁니다. 헬스체크는 redis-postgres는 10초 주기, DRF-FastAPI는 15초 주기로 검사하며, 기동 직후 오류를 막기 위해 start_period 30초를 둡니다.

③ Celery 큐 분리 — 알람이 메트릭에 밀리지 않는다

Celery 워커를 알람(alarm) 큐와 메트릭(metric) 큐로 분리했습니다. 알람 워커는 동시 처리 4(concurrency=4), 메트릭 워커는 1(concurrency=1)로 설정해 별도 프로세스로 격리했습니다. 이렇게 분리하면 메트릭 수집 작업이 물려도 긴급한 알람 처리가 그 뒤에 밀려 지연되는 일이 없습니다. 위험 알람은 사람 안전과 직결되므로 우선순위를 보장한 설계입니다.

④ Redis 영속화 — 재시작해도 데이터가 안 날아간다

Redis를 --appendonly yes (AOF, Append Only File) 모드로 실행합니다. 모든 쓰기 명령을 디스크 파일에 기록해두기 때문에, Redis가 재시작되어도 그 파일을 재생해 데이터를 복원합니다. 그 결과 재시작 후에도 Celery 작업 큐와 실시간 상태 데이터가 유지되어, 처리 중이던 알람 작업이 유실되지 않습니다.

```
ltw17@DESKTOP-H7VS06N:~/diconai$ docker compose exec redis kill 1
sleep 8
docker compose ps redis
NAME                IMAGE             COMMAND                  SERVICE    CREATED      STATUS              PORTS
diconai-redis-1     redis:7-alpine    "docker-entrypoint.s..." redis      30 minutes ago Up 7 seconds (health: starting) 6379/tcp
```

[증빙 11.4-1] Redis 내부 프로세스 크래시 시뮬레이션 및 자동 재시작 확인

```
ltw17@DESKTOP-H7VS06N:~/diconai$ docker compose exec celery-worker-alarm celery -A config inspect active_queues 2>&1 | grep -IE "celery@|routing_key"
-> celery@90711c18c38b: OK
* {'name': 'metric', 'exchange': {'name': 'metric', 'type': 'direct', 'arguments': None, 'durable': True, 'passive': False, 'auto_delete': False, 'delivery_mode': None, 'no_declare': False}, 'routing_key': 'metric', 'queue_arguments': None, 'binding_arguments': None, 'consumer_arguments': None, 'durable': True, 'exclusive': False, 'auto_delete': False, 'no_ack': False, 'alias': None, 'bindings': [], 'no_declare': None, 'expires': None, 'message_ttl': None, 'max_length': None, 'max_length_bytes': None, 'max_priority': None}
-> celery@08b17d46955: OK
* {'name': 'alarm', 'exchange': {'name': 'alarm', 'type': 'direct', 'arguments': None, 'durable': True, 'passive': False, 'auto_delete': False, 'delivery_mode': None, 'no_declare': False}, 'routing_key': 'alarm', 'queue_arguments': None, 'binding_arguments': None, 'consumer_arguments': None, 'durable': True, 'exclusive': False, 'auto_delete': False, 'no_ack': False, 'alias': None, 'bindings': [], 'no_declare': None, 'expires': None, 'message_ttl': None, 'max_length': None, 'max_length_bytes': None, 'max_priority': None}
```

[증빙 11.4-2] Celery alarm/metric 큐 분리 확인

ltw17@DESKTOP-H7VS06N:~/diconai\$ echo "=== kill 전 키 개수 ="
docker compose exec redis redis-cli dbsize && \
echo "=== redis 크래시 ===" && \
docker compose exec redis kill 1 && \
sleep 8 && \
echo "=== 복구 후 상태 + 키 개수 ===" && \
docker compose ps redis --format "{{.Status}}" && \
docker compose exec redis redis-cli dbsize
=== kill 전 키 개수 ===
(integer) 155
=== redis 크래시 ===
=== 복구 후 상태 + 키 개수 ===
Up 7 seconds (health: starting)
(integer) 156

ltw17@DESKTOP-H7VS06N:~/diconai\$ docker inspect diconai-re
--format '{{.Name}}' --restart={{.HostConfig.RestartPolic
echo "---- Redis AOF ----" && \
docker compose exec redis redis-cli config get appendonly
/diconai-redis-1 --restart=unless-stopped
/diconai-drf-1 --restart=unless-stopped
/diconai-fastapi-1 --restart=unless-stopped
--- Redis AOF ---
1) "appendonly"
2) "yes"

[증빙 11.4-3] Redis Kill 전후 키 개수 비교를 통한 데이터 보존 확인

[증빙 10.4-4] Docker restart 정책 및 Redis appendonly 설정 확인

11.9 백업·복구 및 데이터 무결성

데이터 보존은 DataRetentionPolicy와 매일 실행되는 Celery 태스크 (data_retention_task)로 자동 화해, 카테고리별로 원본은 30일, 이력은 90~365일까지만보관하고 오래된 데이터는 자동 삭제하도록 했습니다.

이는 과거 DB가 12GB까지 비대해진 경험을 계기로 도입한 정책입니다. 무결성 측면에서는 PowerData에 UniqueConstraint를 두어 동일 시점 중복 저장을 막고, FK를 PROTECT로 설정해 이력이 있는 센서가 삭제되지 않도록 했으며, PostgreSQL은 pg_dump로 백업합니다.

```
ltw17@DESKTOP-H7VS06N:~/diconai$ docker compose exec drf python manage.py shell -c "
from apps.operations.models.data_retention_policy import DataRetentionPolicy
for p in DataRetentionPolicy.objects.all():
    print(p.data_category, '| raw', p.raw_retention_days, '| hist', p.history_retention_days)
" 2>&1 | grep -v imported

gas_anomaly | raw 30 | hist 365
gas_raw | raw 30 | hist 90
ml_model | raw 90 | hist 90
ml_result | raw 180 | hist 180
position_hist | raw 30 | hist 90
power_agg | raw 30 | hist 365
power_raw | raw 30 | hist 90
app_log | raw 90 | hist 90
integration_log | raw 90 | hist 90
login_log | raw 365 | hist 365
notification | raw 90 | hist 90
system_log | raw 365 | hist 365
```

[증빙 11.9] 백업·무결성